
Course: **EE321 Intro to AI**

Complex Engineering Problem

Name: S. Ali Bakhsh

Reg.: 2023669

IMPORTANT: This is an INDIVIDUAL project

You must run the code yourself.

The exam will ask specific questions about YOUR model's behavior.

If you copy from the internet, you will likely to fail the viva/exam.

Your Must

Build an appropriate AI model that can tell whether a picture contains a **hot dog** or **not a hot dog**.

Learning Objectives

By the end of this project, you will be able to:

- Load and explore image data
- Build an ANN and a CNN in TensorFlow/Keras
- Identify and fix overfitting using Dropout and Data Augmentation
- Plot and interpret training curves
- Test your model on real-world images

Dataset

You will use the **Hot Dog / Not Hot Dog** dataset from Kaggle:

<https://www.kaggle.com/dansbecker/hot-dog-not-hot-dog>

What You Need

- A Google account (for Google Colab)
- The dataset downloaded to your computer (upload to Colab)
- **Submit your overleaf generated PDF by May 07, 2026 by 12:00 PM**

Grading Rubric

Task	Points
Step 2: Data Exploration (answers)	5
Step 3: Visualization (screenshot)	5
Step 5: ANN Model (accuracy + analysis)	10
Step 6: CNN Model (accuracy + comparison)	15
Step 7: Overfitting Fix (before/after plots)	20
Step 8: Real Image Test (screenshot)	10
Step 9: Answers to all questions	20
Step 10: YOLO Extension (extra credit)	5
Total	85 + 5 EC

Step 1: Setup Google Colab

Follow these instructions carefully:

How to Run This Project in Google Colab

1. Go to <https://colab.research.google.com>
2. Click **File** → **New Notebook**
3. Rename it: `HotDog_Classifier_YourName.ipynb`
4. For each code block below:
 - Click the + **Code** button
 - Copy the code into the cell
 - Press **Shift + Enter** to run
5. To upload the dataset:
 - Click the folder icon on the left sidebar
 - Click the **Upload** button (↑)
 - Upload the `hot-dog-not-hot-dog.zip` file
 - Run: `!unzip hot-dog-not-hot-dog.zip`

Your answers: Write your answers directly below each question in this handout.

Step 2: Import Libraries

Run this code in Colab:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import os
import random

print("Libraries loaded successfully!")
print("TensorFlow version:", tf.__version__)
```

1. What TensorFlow version are you using?

2.20.0

Step 3: Load and Explore the Data

Run this code to see how many images you have:

```
# Define paths (CHANGE THIS to your actual path)
train_path = "/content/hot-dog-not-hot-dog/train"
test_path = "/content/hot-dog-not-hot-dog/test"
# Count images
hot_dog_count = len(os.listdir(os.path.join(train_path, "hot_dog")))
not_hot_dog_count = len(os.listdir(os.path.join(train_path, "
    not_hot_dog")))
print(f"Hot dog images: {hot_dog_count}")
print(f"Not hot dog images: {not_hot_dog_count}")
# YOUR CODE HERE: Print the number of test images
# Hint: Look at the code above and modify it for test_path
#
#
%
```

1. How many training images are in each class?

249 images in the "hot_dog" class and 249 images in the "not_hot_dog" class.

2. Is the dataset balanced? Why does that matter?

Yes, the dataset is balanced because both classes have an equal number of samples. This matters because it prevents the model from developing a bias toward a majority class; if one class had more images, the model might learn to always predict that class just to achieve a high accuracy score without learning the features of the images.

Step 4: Visualize the Data

```
# Display 6 random images
plt.figure(figsize=(12, 6))
for i in range(6):
    # Pick random class
    class_name = random.choice(["hot_dog", "not_hot_dog"])
    class_path = os.path.join(train_path, class_name)
    img_name = random.choice(os.listdir(class_path))
    img_path = os.path.join(class_path, img_name)
    # Load and show image
    img = plt.imread(img_path)
    plt.subplot(2, 3, i+1)
    plt.imshow(img)
    plt.title(class_name)
    plt.axis('off')
plt.tight_layout()
plt.show()
# YOUR CODE HERE: Print the shape (height, width, channels) of one
# image. Hint: Use img.shape after loading an image
#
#
```

1. Paste a screenshot of your 6 random images here:

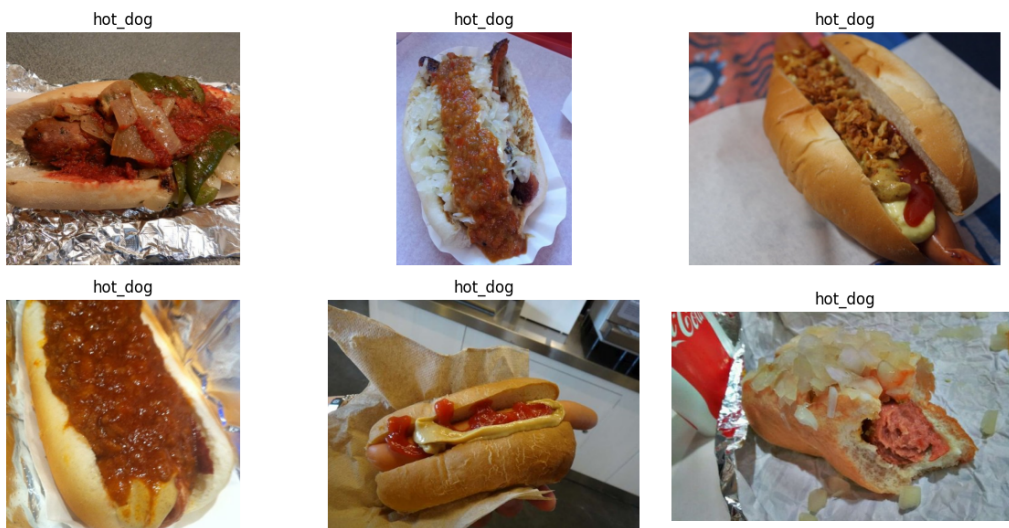


Figure 1: 6 Random images from the dataset

2. What is the shape (height, width, channels) of a typical image? (Use `img.shape`)
The shape of the image is (308, 512, 3). Height, Width and channel respectively.

Step 5: Prepare the Data Generator

```
# Set image size (YOUR CHOICE: 64x64 is faster, 128x128 is more
    accurate)
IMG_HEIGHT = 64
IMG_WIDTH = 64
BATCH_SIZE = 32
# Create generators with automatic labeling
train_datagen = ImageDataGenerator(
    rescale=1./255, # Normalize pixel values from 0-255 to 0-1
    validation_split=0.2 # Use 20% of training data for validation
)

train_generator = train_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary', # Binary classification (hot dog vs not)
    subset='training' # Training set (80% of data)
)

validation_generator = train_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='validation' # Validation set (20% of data)
)
# YOUR CODE HERE: Print the class labels
#
# Hint: Print train_generator.class_indices
```

1. What does class label 0 represent? What does 1 represent?

Based on the generator's class indices, the label **0** represents `hot_dog` and the label **1** represents `not_hot_dog`.

Step 6: Build a Simple ANN (It Will Fail)

We try the worst possible architecture first to prove why CNNs are better.

```
# Build a simple ANN
ann_model = keras.Sequential([
    # Flatten the 2D image into 1D
    layers.Flatten(input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
    # Dense layers
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid') # Output: probability of "
        hot dog"
])
# Compile the model
ann_model.compile(optimizer='adam', loss='binary_crossentropy',
    metrics=['accuracy'])
# Show model summary
ann_model.summary()
# Train for 10 epochs
history_ann = ann_model.fit(train_generator, validation_data=
    validation_generator, epochs=10)
# YOUR CODE HERE: Plot training vs validation accuracy.
#
#
# Hint: Use
# plt.plot(history_ann.history['XXXX'], label='XXXX')
# plt.plot(history_ann.history['XXXX'], label='XXXX')
```

1. What was the final validation accuracy of the ANN?
The final validation accuracy was **0.5816** (or 58.16%).
2. Why does the ANN perform poorly on image data? (One sentence)
The ANN performs poorly because it flattens the 2D image into a 1D vector.
3. Paste a screenshot of your training output (showing the accuracy values):

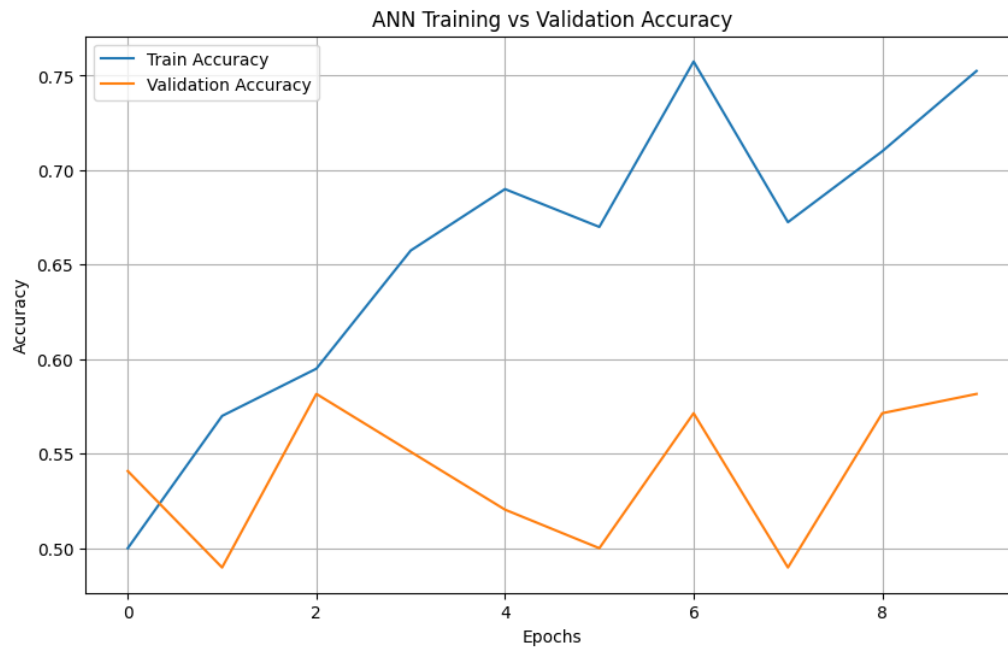


Figure 2: ANN Training vs. Validation Accuracy Plot

Step 7: Build a CNN

Now we use Convolutional and Pooling layers.

```
cnn_model = keras.Sequential([
    # First Conv layer: 32 filters, 3x3 kernel
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(
        IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),
    # Second Conv layer: 64 filters
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    # Third Conv layer (YOUR TURN): Add 64 filters with 3x3 kernel,
    # followed by Max pooling
    # YOUR CODE HERE:
    #
    #
    # Flatten and dense layers
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
# Compile (copy from ANN)
# YOUR CODE HERE:
#
#
#
# Hint: use optimizer = adam, loss = binary_crossentropy, and metrics
```

```

= accuracy

# Train the CNN
history_cnn = cnn_model.fit(train_generator, validation_data=
    validation_generator, epochs=15)

```

1. What was the final validation accuracy of the CNN?

The final validation accuracy of the CNN was **0.5918** (or 59.18%).

2. Compare ANN vs CNN. Which one performed better and why?

The **CNN** performed better because it uses convolutional layers to detect spatial patterns and local features (like edges and shapes) regardless of their position in the image. While the ANN treats an image as a flat list of pixels, the CNN maintains the 2D structure, allowing it to generalize better to the validation set even with a small dataset.

3. Paste the accuracy plot comparing ANN and CNN: (Use the code below to generate the plot)

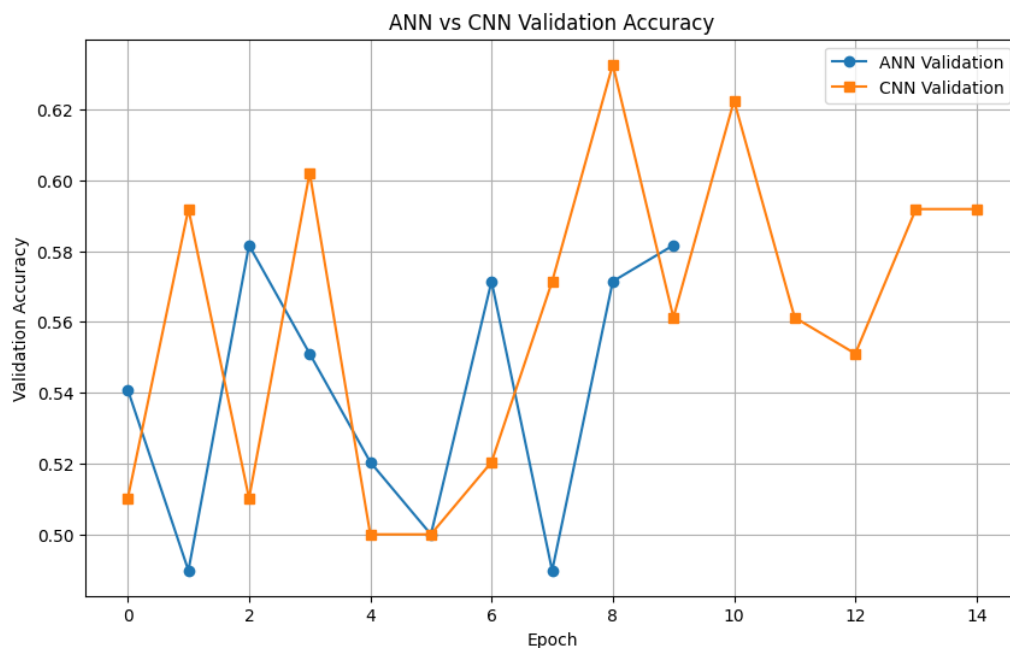


Figure 3: Validation Accuracy Comparison: ANN vs. CNN

```

plt.plot(history_ann.history['val_accuracy'], label='ANN
    Validation', marker='o')
plt.plot(history_cnn.history['val_accuracy'], label='CNN
    Validation', marker='s')
plt.xlabel('Epoch')
plt.ylabel('Validation Accuracy')
plt.legend()
plt.title('ANN vs CNN Validation Accuracy')
plt.grid(True)
plt.show()

```

Step 8: Fix Overfitting (Most Important)

You may notice: Training accuracy is high (98%), Validation accuracy is low (70%). This is OVERFITTING. The model memorized the training images but doesn't generalize.

Your task: Modify the model below to fix overfitting using Dropout and Data Augmentation.

8.1 Fix 1: Add Dropout

```
from tensorflow.keras import regularizers

# Add dropout of 0.25, 0.25, and 0.5 after each MaxPooling layer
# respectively
improved_model = keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(
        IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout( ), # Add dropout

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    # , # Add dropout

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    # , # Add dropout before output
    layers.Dense(1, activation='sigmoid')
])
# Recompile and Train the improved model (after addressing overfitting
# using dropout)
improved_model.compile(optimizer='adam', loss='binary_crossentropy',
    metrics=['accuracy'])

history_improved = improved_model.fit(train_generator, validation_data
    =validation_generator, epochs=15)
```

8.2 Fix 2: Add Data Augmentation

Modify the ImageDataGenerator to create more variations:

```
augmented_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    validation_split=0.2
)

train_augmented = augmented_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='training'
)

# Then retrain the improved model with augmented_datagen
final_model = keras.Sequential([
    # Same architecture as improved_model
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(
        IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])

final_model.compile(optimizer='adam', loss='binary_crossentropy',
    metrics=['accuracy'])

history_final = final_model.fit(train_augmented, validation_data=
    validation_generator, epochs=15)
```

1. Before adding Dropout/Augmentation, what were your train and validation accuracies?

Train: 89.50% Validation: 59.18%

2. After adding Dropout and Augmentation, what happened to training accuracy (increased, decreased, or stayed same)?

The training accuracy **decreased**.

Train: 61.75%

3. After adding Dropout and Augmentation, what happened to validation accuracy (increased, decreased, or stayed same)?

The validation accuracy **increased**.

Validation: 63.27%

Question 8.3

Paste the before/after validation accuracy plot here:

1. Paste the accuracy plot comparing the original CNN and the improved version:

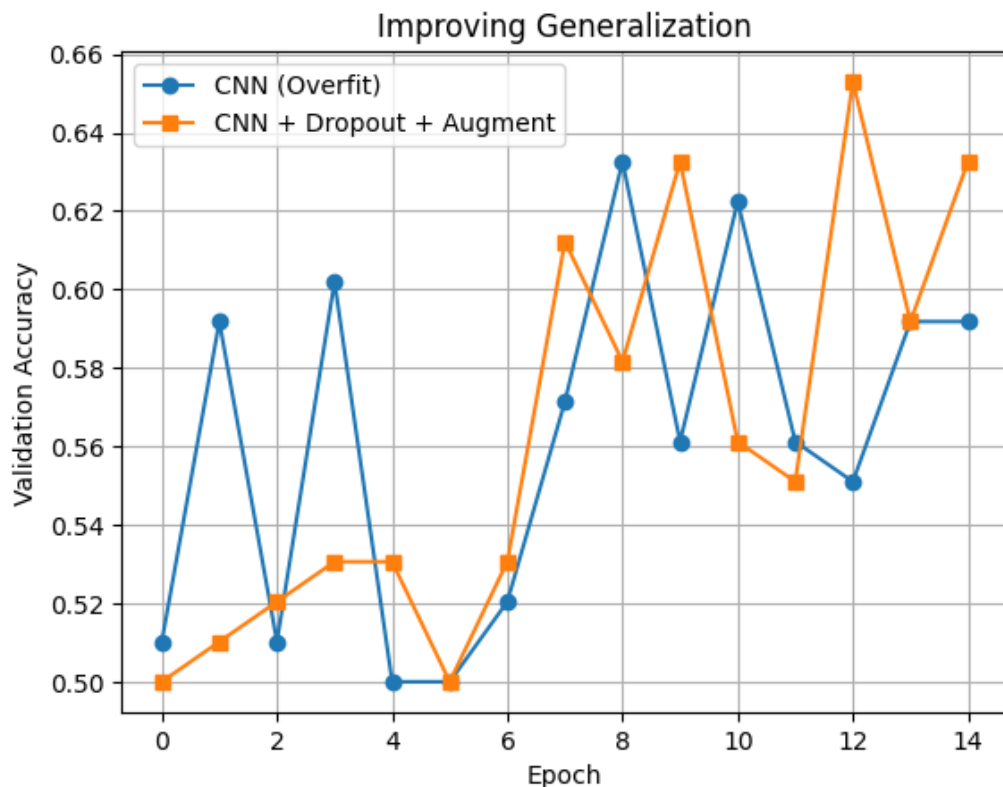


Figure 4: Validation Accuracy Comparison: Original CNN vs. Regularized CNN

2. Use the code below to generate the comparison plot:

```
plt.plot(history_cnn.history['val_accuracy'], label='CNN (
    Overfit)', marker='o')
plt.plot(history_final.history['val_accuracy'], label='CNN +
    Dropout + Augment', marker='s')
plt.xlabel('Epoch')
plt.ylabel('Validation Accuracy')
plt.legend()
plt.title('Improving Generalization')
plt.grid(True)
plt.show()
```

Step 9: Test on Real Images

Find a hot dog image online (or take a photo) and test your model:

```
from tensorflow.keras.preprocessing import image
import requests
from io import BytesIO

def predict_image(img_url):
    # Download image
    response = requests.get(img_url)
    img = image.load_img(BytesIO(response.content), target_size=(
        IMG_HEIGHT, IMG_WIDTH))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0) / 255.0

    # Predict
    prediction = final_model.predict(img_array)[0][0]
    if prediction > 0.5:
        print(f" HOT DOG! (confidence: {prediction:.2f})")
    else:
        print(f" NOT A HOT DOG! (confidence: {1-prediction:.2f})")

    # Show image
    plt.imshow(img)
    plt.axis('off')
    plt.show()

# Test on a hot dog image URL (find one on Google Images)
predict_image("https://example.com/hotdog.jpg") # REPLACE WITH REAL
URL
```

1. Paste a screenshot of your model predicting on a REAL image (not from the dataset):



Figure 5: Final model prediction on an external image.
HOT DOG! (Confidence: 0.63)

2. Did your model ever make a funny mistake (e.g., calling a hamburger a hot dog)? What did it confuse?

No, the model did not confuse other food items with a hot dog during testing. The model was given small dataset and the heavy regularization required to stop overfitting, the model struggled to recognize hot dogs with high confidence even when they were clearly present. It tended to be "uncertain" rather than "confidently wrong" about other food types.

Step 10: Optional Stretch Goal

YOLO Object Detection (Extra Credit)

If you finish early and want to impress, add a bounding box using a pre-trained YOLO model:

```
!pip install ultralytics -q
from ultralytics import YOLO

# Load a tiny pre-trained model
yolo_model = YOLO("yolov8n.pt") # n = nano (fast, small)

# Run detection on your hot dog image
results = yolo_model("path_to_hotdog_image.jpg")

# Show results (it will draw boxes around objects it recognizes)
results[0].show()
```

Extra Credit Question

Does YOLO correctly detect a "hot dog" as a class? What label does it give?

Answer: Yes, the YOLO model correctly detects the "hot dog" class. It successfully identified both hot dogs in the image, providing the specific label **"hot dog"** with confidence scores of **0.92** and **0.90**, respectively.

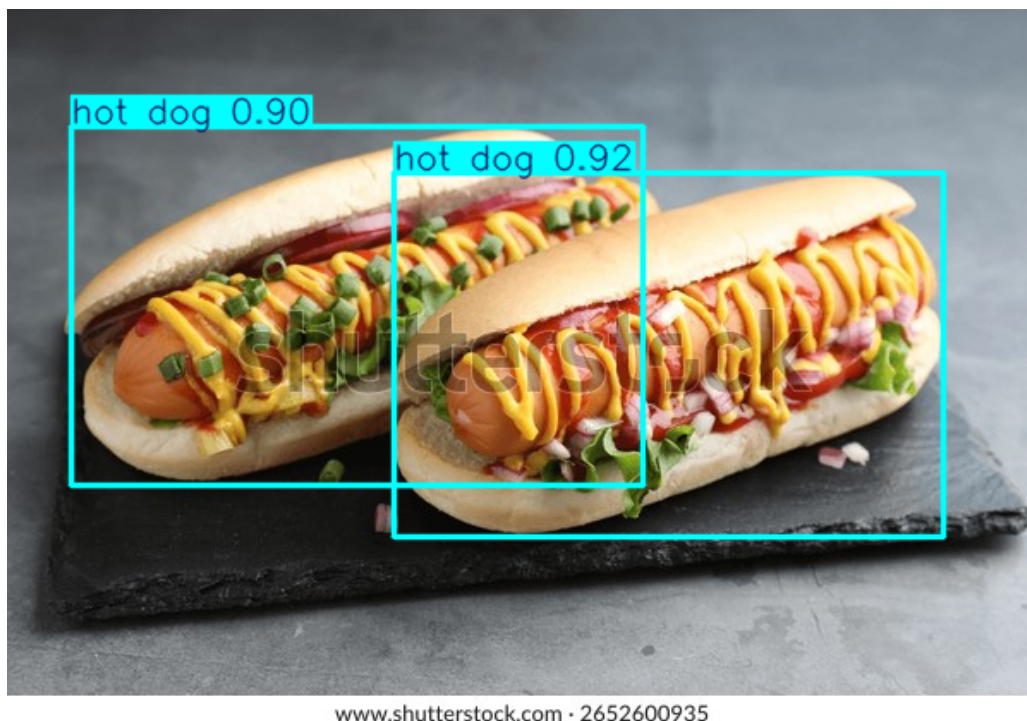


Figure 6: YOLOv8 Object Detection results identifying multiple hot dog instances.

Final Submission Checklist

Step 2: TensorFlow version answer

Step 3: Image counts and balance explanation

Step 4: Screenshot of 6 images + shape answer

Step 6: ANN accuracy + explanation + screenshot

Step 7: CNN accuracy + comparison + plot screenshot

Step 8: Overfitting before/after numbers + plot screenshot

Step 9: Real image prediction screenshot + mistake explanation

Step 10: (Extra credit) YOLO results screenshot

Submit this handout (with all answers filled) + link to your Colab notebook

Remember

The exam will ask you specific questions about YOUR project.

If you run the code yourself, you will know the answers easily.

If you copy from someone else, you will fail the exam.

Good luck and have fun!